

DP Computer Science: B 2.3.1 Code Instructions

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain the importance of **correct instruction order** in programming
- Identify and avoid common programming errors like **infinite loops** and **deadlocks** (where applicable)
- **Analyse** code segments to predict output and identify potential issues

Key Vocabulary

Term	Definition
Instruction	A single command or step in a program that the computer executes
Instruction sequence	The order in which instructions are executed in a program
Sequential execution	Instructions are executed one after another in order
Infinite loop	A loop that never terminates because its condition is always true
Deadlock	A situation where two or more processes are blocked, each waiting for a resource held by the other
Logic error	An error that causes the program to produce incorrect results (but doesn't crash)
Syntax error	An error in the code that violates the language rules

IB ATL Skills

Skill	Description
Thinking	Critical thinking (analysing and evaluating issues); creative thinking (generating novel ideas)
Communication	Exchanging thoughts and information effectively through interaction
Self-Management	Managing time and tasks effectively
Research	Finding, interpreting, judging, and creating information

2. Introduction: The Importance of Instruction Order

What Are Instructions?

Instructions are the individual commands in a program. Programs are sets of instructions that the computer executes in a specific order.

Why Order Matters

text

WHY ORDER MATTERS

Real-World Example: Putting on Shoes and Socks

✔ Correct Order:

1. Put on socks

2. Put on shoes

→ Comfortable and functional!

✘ Incorrect Order:

1. Put on shoes

2. Put on socks

→ Uncomfortable and incorrect!

The same principle applies to programming!

The Importance of Correct Instruction Sequencing

Reason	Description
Correct Output	Instructions must be in the right order to produce the correct result
Program Flow	The sequence determines how the program behaves
Efficiency	Well-ordered instructions are more efficient
Debugging	Understanding order helps find errors quickly
Logic	The logic of the program depends on the order of execution

3. Instruction Order Examples

Example 1: Correct Order

python

```
# Python Example: Correct Order
name = "Alice"
age = 25
print(f"Name: {name}, Age: {age}")
# Output: Name: Alice, Age: 25
```

java

```
// Java Example: Correct Order
String name = "Alice";
int age = 25;
System.out.println("Name: " + name + ", Age: " + age);
// Output: Name: ALice, Age: 25
```

Example 2: Incorrect Order

```
python

# Python Example: Incorrect Order
age = 25
# Error: name is used before it is defined!
print(f"Name: {name}, Age: {age}")
name = "Alice"
# Output: NameError: name 'name' is not defined
```

```
java

// Java Example: Incorrect Order
int age = 25;
// Error: name is used before it is declared!
System.out.println("Name: " + name + ", Age: " + age);
String name = "Alice";
// Output: cannot find symbol
```

Example 3: Logic Order

```
python

# Python Example: Logic Order
x = 5
y = 10

# Correct order
sum = x + y
print(f"Sum: {sum}") # Output: Sum: 15

# Incorrect order (x not yet defined)
# print(f"Sum: {sum}")
# sum = x + y # ❌ Error
```

4. Common Programming Errors

Infinite Loops

| An infinite loop is a loop that never terminates because its condition is always true.

text

INFINITE LOOP EXAMPLE

```
while True:
    print("This will run forever!")
```

⚠️ The condition is always True, so the loop never ends.

Infinite Loop Examples

python

```
# Python: Infinite Loop (missing increment)
i = 0
while i < 10:
    print(i)
    # Missing: i = i + 1
    # i never changes, so the loop runs forever!

# Python: Infinite Loop (condition always true)
while 1 == 1:
    print("Always true!")
    # The condition is always True
```

java

```
// Java: Infinite Loop (missing increment)
int i = 0;
while (i < 10) {
    System.out.println(i);
    // Missing: i++;
    // i never changes, so the loop runs forever!
}

// Java: Infinite Loop (condition always true)
while (true) {
    System.out.println("Always true!");
    // The condition is always True
}
```

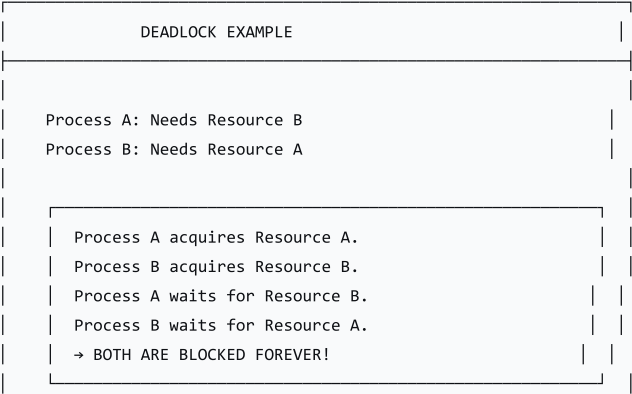
How to Avoid Infinite Loops

Strategy	Description
Check Loop Conditions	Ensure the condition will eventually become false
Update Loop Variables	Always increment/decrement loop variables
Use Break Statements	Add a condition to break out of the loop
Test with Small Inputs	Test loops with small values first

Deadlocks

A deadlock occurs when two or more processes are blocked, each waiting for a resource held by the other.

text



- ⚠ Both processes are waiting for each other.
- ⚠ Neither can proceed.

Analogy: Two people trying to pass each other in a narrow hallway. Each is waiting for the other to move first.

Incorrect Output

Incorrect output occurs when a program runs without errors but produces the wrong result.

Cause	Description
Logic Errors	The program logic is flawed
Incorrect Variable Usage	Using the wrong variable
Data Type Issues	Mixing incompatible data types
Off-by-One Errors	Loop one iteration too many or too few

5. Code Analysis Activities

Activity 1: Predict the Output

Python Code:

```
python
x = 10
y = 5
result = x - y
print(result)
y = 10
result = x + y
print(result)
```

What is the output?

Line	x	y	result	Output
1	10	-	-	-
2	10	5	-	-
3	10	5	5	-
4	10	5	5	5
5	10	10	5	-
6	10	10	20	-
7	10	10	20	20

Final Output: 5 then 20

Activity 2: Identify the Error

```
python

# What is wrong with this code?
total = 0
for i in range(1, 10):
    total = total + i
# Missing print statement!
```

Error: The program calculates the sum but never prints it.

```
python

# Corrected code:
total = 0
for i in range(1, 10):
    total = total + i
print(total) # Output: 45
```

Activity 3: Fix the Infinite Loop

```
python

# This code has an infinite loop. Fix it!
count = 1
while count <= 10:
    print(count)
# Missing: count = count + 1
```

```
python

# Fixed code:
count = 1
while count <= 10:
    print(count)
    count = count + 1 # Added increment
```

6. Debugging Strategies

Common Debugging Techniques

Technique	Description
Trace Tables	Track variable values step by step
Print Statements	Insert <code>print()</code> statements to check values
Breakpoints	Pause execution at specific lines
Step-by-Step Execution	Run code line by line
Rubber Duck Debugging	Explain code to someone (or something)

Example: Using Print Statements

```
python
```

```
def calculate_average(numbers):
    total = 0
    print(f"Initial total: {total}") # Debug
    for num in numbers:
        total += num
        print(f"Added {num}, total is now {total}") # Debug
    average = total / len(numbers)
    print(f"Average: {average}") # Debug
    return average
```

7. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	What are instructions? Why order matters (shoes/socks analogy)
Instruction Order	20 min	Code examples (correct vs. incorrect order); Activity 1
Common Errors	20 min	Infinite loops, deadlocks, incorrect output; Activity 2
Conclusion	10 min	Key takeaways; further exploration

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of instruction order and common errors
Code Review	Students review each other's code

9. Differentiation

Level	Strategy
Support	Provide scaffolded examples; use visual aids
Challenge	Explore more complex errors; advanced debugging techniques

10. Resources

- Presentation (Slide Deck)
- eBook (B2.3.1 with examples)
- Worksheets: Activity 1 & 2
- Worksheet Answers

11. Summary: Key Takeaways

Concept	Key Point
Instruction Order	The order of instructions determines the program's behaviour
Infinite Loops	Occur when loop conditions never become false
Deadlocks	Processes waiting for each other's resources
Incorrect Output	Caused by logic errors, wrong variables, etc.
Debugging	Use trace tables, print statements, and breakpoints

text

KEY REMINDERS

✔

Order matters—instruction sequence determines output

✔

Infinite loops never terminate

✔

Deadlocks occur when processes wait for each other

✔

Incorrect output ≠ no output

✔

Use debugging techniques to find errors

“The sequence of instructions determines the behaviour of a program—one small mistake in order can change everything.”